Baldur: Whole-Proof Generation and Repair with Large Language Models

Emily First
University of Massachusetts
Amherst, MA, USA
efirst@cs.umass.edu

Markus N. Rabe
Google, Inc.
CA, USA
mrabe@google.com

Talia Ringer
University of Illinois Urbana-Champaign
IL, USA
tringer@illinois.edu

Yuriy Brun
University of Massachusetts
Amherst, MA, USA
brun@cs.umass.edu

arXiv:2303.04910v2 [cs.LG] 16 Mar 2023

ABSTRACT

Formally verifying software properties is a highly desirable but labor-intensive task. Recent work has developed methods to automate formal verification using proof assistants, such as Coq and Isabelle/HOL, e.g., by training a model to predict one proof step at a time, and using that model to search through the space of possible proofs. This paper introduces a new method to automate formal verification: We use large language models, trained on natural language text and code and fine-tuned on proofs, to generate whole proofs for theorems at once, rather than one step at a time. We combine this proof generation model with a fine-tuned repair model to repair generated proofs, further increasing proving power. As its main contributions, this paper demonstrates for the first time that: (1) Whole-proof generation using transformers is possible and is as effective as search-based techniques without requiring costly search. (2) Giving the learned model additional context, such as a prior failed proof attempt and the ensuing error message, results in proof repair and further improves automated proof generation. (3) We establish a new state of the art for fully automated proof synthesis. We reify our method in a prototype, Baldur, and evaluate it on a benchmark of 6,336 Isabelle/HOL theorems and their proofs. In addition to empirically showing the effectiveness of whole-proof generation, repair, and added context, we show that Baldur improves on the state-of-the-art tool, Thor, by automatically generating proofs for an additional 8.7% of the theorems. Together, Baldur and Thor can prove 65.7% of the theorems fully automatically. This paper paves the way for new research into using large language models for automating formal verification.

1 INTRODUCTION

Formal software verification — proving software correctness and other properties — is one of the most challenging tasks software engineers can undertake. It is highly effective at producing high quality software. For example, CompCert, a C compiler verified using the Coq interactive theorem prover [81], was the only compiler on a list including the ubiquitous GCC and LLVM, in which a comprehensive study found no bugs [97]. Similarly, the seL4 project resulted in an highly reliable operating system microkernel [40]. However, the cost of manual formal verification — writing the proofs — is often prohibitive. For example, the proof of the C compiler is more than three times as long as the compiler code itself [47]. As a result, recent research has focused on automated proof synthesis, which can lead to fully automating formal verification.

There are two promising approaches for automating proof synthesis. The first is to use *hammers*, such as Sledgehammer [64] for the Isabelle proof assistant. Hammers iteratively apply known mathematical facts using heuristics. The second is to use search-based *neural theorem provers*, such as DeepHOL [4], GPT-f [66], TacticZero [91], Lisa [34], Evariste [42], Diva [20], TacTok [22], and ASTactic [96]. Given a partial proof and the current *proof state* (which consists of the current goal to prove and the list of known assumptions), these tools use neural networks to predict the next individual *proof step*. They use the *proof assistant* to evaluate the proposed next proof steps, which returns a new set of proof states. Neural theorem provers rely on diverse neural architectures, such as Wavenet [4, 84], graph neural networks [62], short long-term memory models [20], and language models with the transformer architecture [27, 66].

In this paper, we propose Baldur, a different, simpler approach to proof synthesis. We show that using large language models (LLMs), fine-tuned on proofs, can produce entire proofs for theorems. LLMs are scaled-up transformer models trained on a large amount of text data, including natural language and code, that have proven to be remarkably effective across a wide variety of applications, including question answering, and text and code generation [7, 14]. Here, we show their remarkable effectiveness for whole proof generation.

The main contributions of our work are:

- We develop Baldur, a novel method that generates whole formal proofs using LLMs, without using hammers or computationally expensive search.
- We define a proof repair task and demonstrate that repairing incorrectly generated proofs with LLMs further improves Baldur's proving power when the LLM is given access to the proof assistant's error messages.
- We demonstrate empirically on a large benchmark that Baldur, when combined with prior techniques, significantly improves the state of the art for theorem proving.

We design Baldur to be able to work with any LLM internally, but we evaluate our implementation using two versions of Minerva [48], one with 8 billion parameters and another with 62 billion parameters. By contrast, existing tools that use (L)LMs for theorem

proving, either predict individual proof steps [27, 33, 34], or rely on few-shot prompting and require the existence of natural language proofs as hints [35].

We evaluate Baldur on the PISA dataset [34] of Isabelle/HOL theorems and their proofs used in recent state-of-the-art Isabelle/HOL proof synthesis evaluations [33, 34]. The dataset consists of 183K theorems, of which we use 6,336 for measuring effectiveness. Our evaluation answers the following research questions:

RQ1: How effective are LLMs at generating whole proofs?
LLMs outperform small-model-driven search-based methods. Baldur (without repair) is able to generate whole proofs for 47.9% of the theorems completely automatically, whereas search-based approaches prove 39.0% [33].

RQ2: Can LLMs be used to repair proofs?
LLMs can repair proofs, including their own erroneous proof attempts. Baldur proves an additional 1.5% of the theorems when given access to a previous erroneous proof attempt and the error messages produced by the proof assistant, even when controlling for the computational cost of the additional inference. The error message is crucial for this improvement.

RQ3: Can LLMs benefit from using the context of the theorem?
In-context learning is remarkably effective for LLM-based theorem proving. With context, Baldur proves 47.5% of the theorems, but only 40.7% without context for the same model size.

RQ4: Does the size of the LLM affect proof synthesis effectiveness?
Larger LLMs do perform better, suggesting that our approach will continue to improve with further developments in LLM research.

RQ5: How do LLMs compare to other state-of-the-art proof generation methods?
Baldur complements state-of-the-art approaches by proving theorems they do not. Together with Thor [33], a tool that combines a learned model, search, and a hammer, Baldur can prove 65.7% of the theorems, whereas Thor alone proves 57.0%. These findings suggest that LLM- and search-based methods' ideas complement each other and can work together to further improve the automation of formal verification. An ensemble of 10 different fine-tuned Baldur models proves 58.0%.

By leveraging LLMs, Baldur simplifies the proof synthesis pipeline, greatly reducing the complexity and cost of the fine-grained interaction between the prediction model and the proof assistant that search-based methods require. This reduction enables us to leverage the power of LLMs, which would be prohibitively computationally expensive if synthesis required as many LLM queries as search-based methods. Further, those calls would require re-encoding with each step the additional information the LLM might need, whereas our approach allows us to make a single call and process the context only once, sampling multiple proofs of multiple proof steps, at once.[1] Overall, our study strongly suggest that LLMs are a very promising direction of research for automating formal verification, and identifies several new avenues for future explorations.

[1] Alternatively path advanced caching strategies in the prediction servers of large language models could address this problem. This is beyond the scope of our work.

2 THE BALDUR APPROACH

Prior approaches to proof synthesis employ a neural model to predict the next *proof step* given the current *proof state*. The proof step predictions then guide a search strategy, such as best-first search or depth-first search. Throughout the search, the proof assistant needs to check each proof step prediction to determine whether it is valid. This means that existing proof synthesis tools require a tight interaction between the neural network and the proof assistant. As we move to using LLMs, this results in complex systems, as LLMs need to run on specialized hardware (GPUs or TPUs), while proof assistants run on CPUs.

We explore a simpler, yet effective method: fine-tuning LLMs to generate complete proofs. This simplification avoids the fine-grained interaction between neural model and the proof assistant, allowing us to run the jobs of generating proofs and checking completely separately. Besides reducing complexity, this can also improve efficiency, because (1) it enables us to use large batch sizes, which can significantly improve hardware utilization during inference (cf. [67]), and (2) when providing additional context to the model, the context now does not have to be reprocessed for each proof step, but only once per proof.

We fine-tune LLMs on proof data to generate entire proofs and explore the impact of giving the LLMs additional information. Our approach and implementation include the following:

- We fine-tune an LLM to generate an entire proof given only the theorem statement. We call this model the *proof generation model* (Section 2.1).
- We provide a model a proof attempt that did not check along with the corresponding *error message* from the proof assistant so that the model may attempt to find a better proof. We call this model the *proof repair model* (Section 2.2).
- We provide text from the same *theory file* that the problem was taken from. We add only the lines from the theory file that immediately precede the theorem we want to prove. We call this added information the *theory file context* and we add it to the proof generation model (Section 2.3).
- The LLM that we fine-tune at the core of all of this is Minerva [48], which is pretrained on a mathematics corpus. We describe our Baldur-specific implementation details for how we use this model (Section 2.4).

These fine-tuned LLMs and their interaction with the Isabelle proof assistant make up our tool Baldur. This section details the Baldur approach, which includes creating training datasets and leveraging LLMs to generate and repair proofs.

2.1 Proof Generation

Existing proof generation methods using neural models generate the proof one step at a time. In contrast, our approach generates the entire proof, as illustrated with a single example in Figure 1. We use only the theorem statement as input to our *proof generation model*. We then sample a proof attempt from this model and perform proof checking using Isabelle. If Isabelle accepts the proof attempt without an error, then we have proven the theorem. Otherwise, we can try sampling another proof attempt from the proof generation model. Explicitly, the input and output of our proof generation model is as follows:

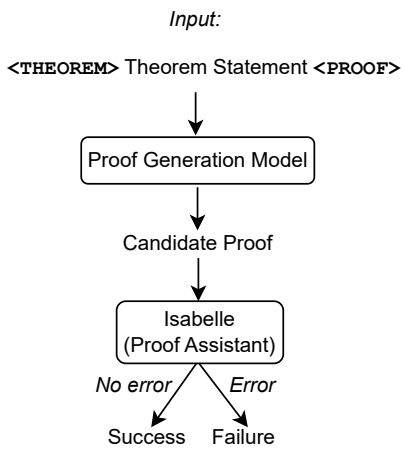

Figure 1: An example of using the proof generation model to generate a proof.

- **Input:** theorem statement.
- **Output:** candidate proof.

Example. To illustrate the power of the proof generation approach in our tool Baldur, we first consider, as an example, the theorem `fun_sum_commute`.

```
lemma fun_sum_commute:
  assumes "f 0 = 0" and "∧x y. f (x + y) = f x + f y"
  shows "f (sum g A) = (Σa∈A. f (g a))"
```

The theorem states that for an additive function f where $f(0) = 0$, and an arbitrary function g, applying f on the sum of the set resulting from applying g on each element in a given set is equal to the sum of applying g followed by f to each element in that set. This theorem is from a project in the Archive of Formal Proofs called Polynomials, specifically in the file `Utils.thy`.

The human-written proof distinguishes between two cases: when the set is finite and when it is not. Induction is used for the finite set case.

```
proof (cases "finite A")
  case True
    thus ?thesis
    proof (induct A)
      case empty
      thus ?case by (simp add: assms(1))
    next
      case step: (insert a A)
      show ?case by (simp add:
        sum.insert[OF step(1) step(2)]
        assms(2)
        step(3))
    qed
  next
    case False
      thus ?thesis by (simp add: assms(1))
qed
```

If we were to derive a training example from this example, the input would be theorem statement and the target would be this human-written proof.

Our tool Baldur, using the proof generation model, is able to generate the following correct proof for this statement.

```
by (induct A rule: infinite_finite_induct)
(simp_all add: assms)
```

Baldur recognizes that induction is necessary and applies a special induction rule called `infinite_finite_induct`, following the same overarching approach as the human-written proof, but much more succinctly. It is interesting to note that Sledgehammer, the hammer for Isabelle, cannot prove this theorem by default, as it requires induction.

Training Data Creation. To train the proof generation model, we construct a new proof generation dataset. Existing datasets for training models in neural theorem provers contain examples of individual proof steps. Each training example includes, at minimum, the proof state (the input) and the next proof step to apply (the target). Given a dataset that contains individual proof steps, we want to create a new dataset so that we can train models to predict entire proofs at once. So we extract the proof steps of each theorem from the dataset and concatenate them to reconstruct the original proofs. We use this data to generate training examples for the proof generation model, where the input consists of the theorem statement and the target consists of the proof.

In particular, this means that we drop the *proof states* from the dataset, which make up most of the text in the dataset. We argue that for Isabelle proofs this is not necessarily a problem, as Isabelle uses a declarative proof language that is designed to be human-readable. This is in contrast to other proof assistants, such as Coq, where the proofs are typically written in a procedural style that is not easy to interpret for humans without using the proof assistant to generate the intermediate proof states.

Inference. We fine-tune an LLM on our data to predict the entire proof given only a theorem statement. To synthesize a proof using the fine-tuned LLM, we provide a potentially unseen theorem statement and sample a fixed number of sequences (typically 16 or 64) from the language model. We tune the sampling temperature from a small set (between 0.0 and 1.4 in increments of 0.2), which is a multiplicative factor on the log probabilities of the distribution of tokens sampled in each step.

Proof checking. After sampling proofs from the model, we check all of them with the proof assistant. This means that we first load the context in which the theorem was originally proven and then replace the original proof of the theorem with the one we sampled from the model. If Isabelle accepts any of the sampled proofs, we report the theorem as proven.

2.2 Proof Repair

If a proof is not accepted, Isabelle returns an error message that is intended to help humans with debugging their proof script. Existing proof generation methods, however, have no way to leverage error messages.